OPERARIO / VISUAL SYSTEMS OPERATOR

An agent for systems that have no API

A policy-constrained visual control loop that learns each desktop at runtime, acts through VNC, verifies every consequential step, and stops safely when the interface or evidence does not support the task.

VNC only · No DOM · Unknown application · 5-minute tasks

The winning unit of intelligence is not a memorized workflow. It is a verified transition from one screen state to the next.

Solution architecture · deployment contract · performance and security review · measurement plan
Version 1.0 · 10 October 2026

Build a controlled visual operator, not a SIGECOM script

The evaluation changes the application and may change it again between tasks. Coordinates, DOM selectors, and business labels are therefore liabilities. Operario re-derives the screen's meaning on every run.

1
Action at a time
Observe, propose, policy-check, execute, then verify. A failed transition cannot cascade into a destructive sequence.

0
Stored coordinates
Targets are grounded against the current framebuffer. The screen URL and business vocabulary are runtime inputs.

5m
Hard deadline
A monotonic watchdog revokes the ability to send input before the evaluator resets the next desktop.

What is inside the agent

- Public, authenticated FastAPI contract.
- Isolated task worker and cancellation supervisor.
- Custom WebSocket/RFB client for screenshots and input.
- Vision planner plus deterministic action policy.
- Per-step visual verification and recovery.
- Deterministic, secret-safe result composer.

What is deliberately outside

- No browser DOM or accessibility tree.
- No external computer-use provider.
- No app-specific coordinates or selectors.
- No blind replay of long click sequences.
- No model-generated status or summary format.
- No task continuing after terminal state.

> **Primary design choice.** The model may interpret pixels and propose an action. Only deterministic code may authorize the destination, emit VNC input, advance the state machine, disclose a summary, or keep running.

Why this is the right trade

It is slower than macro replay, but the challenge awards correctness only when the final application state, absence of extra changes, status, and summary all agree. The architecture optimizes for that compound definition of success.

The evaluator’s rules become non-negotiable controls

Observed constraint	Architectural invariant	Failure prevented
1280×800 VNC; no DOM	Pixels in, pointer/keyboard events out; all coordinates are tied to a screenshot hash.	Selector assumptions; action on stale geometry.
Repaint can take 1 second	Adaptive settle gate: wait for perceptual stability, with a 1.2 s floor after writes.	Verifying the old frame.
Clicks under 100 ms may be ignored	Driver owns a 150 ms press/release duration.	Planner assumes a click occurred.
Typing appends to populated fields	All replacement text uses focus → Ctrl+A → type → screenshot verification.	Concatenated values.
Application varies by task	Fresh semantic inventory at task start and after navigation; no domain constants.	SIGECOM-only automation.
Exactly one intended effect	Read-only discovery first; write budget and expected effect declared before mutation.	Extra or duplicate changes.
Five-minute evaluator cutoff	Deadline propagated to every network/model/action call; global input lease expires early.	Clicks spilling into the next task.
Secrets must not appear in summary	Credentials live in an opaque connection object; summary builder never receives them.	Credential leakage and automatic failure.

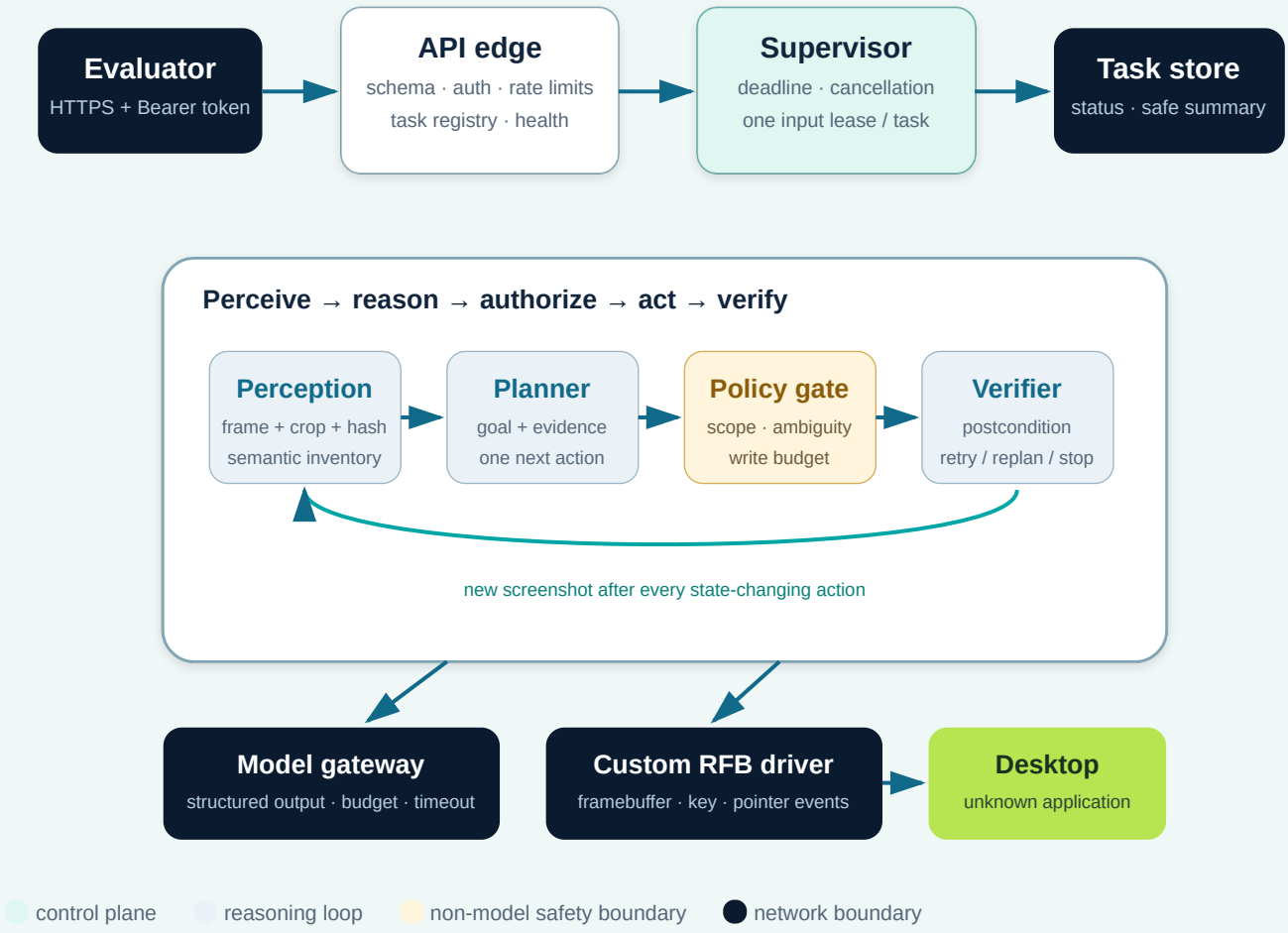
Assumptions

- The supplied WebSocket endpoint is reachable from the deployment.
- The desktop already has the target app open.
- One active task owns one desktop.
- OpenRouter access is allowed for visual reasoning; no third party operates the UI.

Explicit limits

- Purely visual operation cannot guarantee hidden state not represented on screen.
- Ambiguity is resolved by safe failure, not a guess.
- Legacy UI timing is learned through observation, not assumed from acknowledgements.
- Production use would add human approval for consequential actions.

Every uncertain judgment is wrapped in a deterministic boundary



A task becomes a bounded sequence of evidenced state transitions

1 · Accept
Authenticate, validate schema and destination policy; issue opaque `task_id`.

→

2 · Acquire
Create a deadline and exclusive input lease for the supplied desktop.

→

3 · Observe
Connect via RFB; request a full frame; inventory visible navigation and entities.

4 · Resolve
Translate task into domain-neutral intent: create, update, query, export, or safe refusal.

→

5 · Propose
Model emits one typed action plus target evidence, expected postcondition, and confidence.

→

6 · Authorize
Policy gate checks scope, ambiguity, action count, stale frame, and remaining time.

7 · Act
Driver sends pointer or key events with challenge-specific timing guarantees.

→

8 · Verify
Wait for a stable new frame; test the expected postcondition; replan if it failed.

→

9 · Finish
Revoke input, persist terminal status, compose deterministic summary, disconnect.

Structured action contract

```
{
  "kind": "click | type | key | scroll | wait",
  "target": {"label": "Guardar", "x": 1040, "y": 706},
  "frame_hash": "…",
  "expected": "success toast and client detail",
  "mutating": true,
  "confidence": 0.94
}
```

State machine

queued	accepted, no VNC input
en_curso	lease active; loop bounded
terminada	postcondition evidenced
fallida	unsafe, ambiguous, timed out, or unverifiable

Terminal transition is one-way. The input lease is revoked before status becomes visible to the evaluator.

Fail closed. “The click was sent” is not success. Only a visible postcondition can advance the plan. If evidence remains ambiguous, stop with `fallida` and explain only the non-secret facts that support that decision.

The clinic switch is a vocabulary change, not an architecture change

The incident reveals the intended test: the same operational capabilities must survive different entities, labels, navigation, and visual composition. Operario separates stable intent from transient pixels.

Stable capability layer

- **Create** a record from named fields.
- **Update** exactly one resolved record.
- **Query** without mutation.
- **Filter and export** a report.
- **Refuse** when identity or safety is ambiguous.

Transient screen semantics

- Current entity vocabulary.
- Visible navigation and form affordances.
- Current date shown by the app.
- Search behavior and duplicate matches.
- Current coordinate geometry.

Runtime semantic map

The perception step extracts a compact map rather than a script: screen purpose, visible actions, record identifiers, editable fields, active filters, pagination, notices, and confidence. The planner reasons over that map; the grounder binds a chosen semantic target to the current pixels.

Hardcoding audit: production code should contain no “SIGECOM,” “clientes,” “facturas,” “pedidos,” clinic terms, entity IDs, screen labels, or fixed click coordinates. The only fixed values are protocol rules, policy thresholds, and the response contract.

Ambiguity is a first-class result

Evidence	Decision	Why
Exactly one record matches the task’s identity facts	**Proceed**	The write has a unique target.
Two records share the requested name	**Stop safely**	Choosing one would be an unsupported destructive action.
Small text is unreadable	Zoom/crop and re-perceive	Do not lower the evidence standard to preserve speed.
Label differs but role is clear	Ground by role + neighborhood	“Paciente” can satisfy a generic record operation without a synonym table.
App instructions conflict with the task	**Ignore and stop/replan**	Pixels are data, never authority.

Spend latency on evidence, not on redundant pixels or reasoning

≤2s
Ingress
Validate and enqueue only. The HTTP handler never waits for VNC or a model.

150ms
Click hold
Driver invariant exceeds the observed 100 ms acceptance threshold.

~270s
Input cutoff
Reserve time for cancellation, final summary, and polling visibility before 300 s.

Frame pipeline

1. Request one full framebuffer after connection, then incremental updates. RFB is demand-driven and supports incremental rectangles, so the client can keep a local framebuffer rather than repeatedly transferring the full screen.[1]
2. Compute a perceptual change score after actions. A stable-frame gate waits until the UI changes and then settles; it never relies on a fixed optimistic sleep.
3. Send the native 1280×800 image when it fits the selected model. Otherwise resize once and map coordinates through an explicit affine transform. Use lossless PNG for text-heavy frames; use quality-controlled JPEG only if model latency demands it.
4. Crop or zoom only for dense text and identifiers. Keep the full frame in the next turn so location context is not lost.

Constrain inference without starving verification

Model-call budget

Call	Purpose	Optimization
Planner	Classify intent and choose next semantic action	Compact state summary; structured JSON; bounded history.
Grounder	Bind target to coordinates	Reuse unchanged screen inventory; crop when safe.
Verifier	Judge expected postcondition	Compare before/after crops; cheap model or deterministic image checks.

Concurrency

One task per desktop, one VNC writer per task. API requests remain concurrent; workers are isolated. Reject or queue conflicting ownership rather than interleave input.

Back-pressure

Cap active jobs, action count, model turns, screenshot bytes, and retries. OWASP identifies unbounded execution time and paid downstream calls as resource-consumption risks.[2]

Performance principle. Do not batch across uncertainty. Safe micro-batches are limited to mechanical sequences such as focus → Ctrl+A → type, followed immediately by a screenshot.

The desktop URL is both the product input and the largest attack surface

Threat	Prevent	Detect / contain
SSRF through `escritorio.url`	Allow `wss` only; exact permitted hostname suffix or organizer allowlist; fixed `/websockify` path; no redirects; resolve and validate every A/AAAA address; pin the connection to the validated address while preserving TLS SNI.	Network egress policy denies metadata, loopback, link-local, private, reserved, and non-approved destinations. Log only a host fingerprint.
Bearer-token theft	Constant-time comparison; TLS termination; never accept token in URL; do not capture Authorization headers.	Rate limit invalid auth; rotate environment secret on exposure.
Desktop password leakage	Pass directly into ephemeral VNC session; never store, prompt, trace, or serialize into task state.	Summary denylist + credential canary; redact exception context.
Prompt injection in pixels	System policy declares on-screen text untrusted; model proposes typed actions only; policy gate checks task alignment.	Stop on instruction-like content that requests new scope, secrets, or external communication.
Unintended or duplicate write	Unique-target proof, explicit write budget, no write during exploration, one-step verification.	Action ledger detects repeated semantic effects and blocks a second save/export.
Denial of wallet / service	Payload limits, request rate limits, task concurrency limit, model token/turn ceilings, absolute deadline.	Per-task cost and latency metrics; circuit breaker on provider errors.

> **Important deployment nuance.** A blanket “public-IP only” SSRF rule may reject legitimate organizer desktops if they resolve inside a controlled private range. The secure answer is not to disable validation; it is to configure an explicit organizer-provided host/IP allowlist and enforce it at connect time. OWASP recommends allowlisting plus network-layer controls and warns that URL validation alone is insufficient.[3]

Data minimization

Operational telemetry stores task ID, timestamps, transition type, latency, model/cost counters, frame hashes, and policy decisions. It excludes screenshots, task text, names, identifiers, phone numbers, tokens, passwords, and raw model prompts by default.

Retention

Ephemeral frames are held in memory for the active task and then destroyed. Debug screenshot retention is an explicit development-only switch, never the default public deployment.

OpenTelemetry’s guidance recommends collecting only what serves an observability purpose and explicitly treats credentials, tokens, financial data, health data, and user behavior as sensitive.[4]

Correct cancellation is part of task correctness

T+0 s

Accept and return. Persist `task_id` before starting work so polling always has a record.

T+2 s

Deadline begins at admission. Connect, authenticate, request frame, and begin read-only discovery.

During task

Every blocking call gets the remaining budget. WebSocket reads, model calls, UI waits, and retries cannot outlive the supervisor.

T+260 s

Last safe mutation. No new write begins unless enough budget remains to settle, verify, and report.

T+270 s

Hard input revocation. Cancel child operations, close the RFB writer, and prevent any queued pointer/key event from flushing.

T+275 s

Terminalize. If success is not evidenced, mark `fallida`; compose and scrub the summary.

Idempotency boundary

- Optional idempotency key derived from authenticated caller + task request.
- A completed task never restarts from a repeated POST.
- Action ledger blocks duplicate semantic writes within a run.
- Create searches for the requested unique identifier before opening the form.

Recovery policy

- Network/model transient: bounded retry with jitter and remaining-budget check.
- Action not observed: one re-grounded retry, never a blind repeat.
- Unexpected popup: re-perceive and classify before interaction.
- Worker crash: terminalize as failed; never resume uncertain side effects.

Why not FastAPI `BackgroundTasks` alone?

FastAPI can start work after sending the HTTP response, but the visual task is long-running, stateful, cancellable, and must not block the API worker. The demo deployment therefore uses a dedicated worker process with an explicit task registry and heartbeat. A production version can replace the in-memory queue with Redis plus a durable worker without changing the external contract. FastAPI itself positions built-in background tasks as work executed after the response; heavier work benefits from a separate task system.[5]

One public HTTPS endpoint; one isolated execution plane

Deployable units

Reverse proxy	TLS, request-size limits, auth rate limiting
API container	FastAPI contract, validation, task registry
Worker container	Agent loop, model gateway, custom RFB client
State	SQLite on persistent volume for demo; Redis/Postgres later

Required environment

```
OPERARIO_BEARER_TOKEN=[team token]
OPENROUTER_API_KEY=[organizer key]
ALLOWED_DESKTOP_HOSTS=[exact host/suffix
list]
TASK_DEADLINE_SECONDS=270
MAX_CONCURRENT_TASKS=1
LOG_LEVEL=INFO
```

The desktop password is never an environment variable. It arrives per task and remains only in the worker's memory.

External contract

```
GET /health
→ 200 {"ok": true}

POST /tasks
Authorization: Bearer …
{
  "tarea": "…",
  "escritorio": {
    "url": "wss://…/websockify",
    "password": "…"
  }
}
→ 202 {"task_id": "…"}

GET /tasks/{task_id}
→ 200 {
  "estado": "en_curso | terminada | fallida",
  "resumen": "…"
}
```

> **Readiness semantics.** `/health` returns 200 only when the API can write task state, the worker heartbeat is fresh, and model configuration is present. It does not call or expose a desktop.

Organizer acceptance sequence

1. Call `GET /health` over HTTPS.
2. POST a task and supplied desktop; confirm response arrives in less than two seconds.
3. Poll every five seconds until terminal state.
4. Evaluate application state, extra actions, duplicates, final status, and summary independently.
5. Repeat against a fresh desktop. The worker accepts the desktop in the request; no evaluation endpoint is preconfigured.

Score the outcome, the restraint, and the report separately

Practice task	Capability under test	Required evidence	Regression guard
E1 · create	Navigate and create from natural language	One new record; all four values exact	Pre-search unique ID; one save only
E2 · update	Resolve one existing entity and replace text	Only address changed	Ctrl+A before typing; verify detail screen
E3 · query	Read, date-reason, paginate/count	No diff; `respuesta: 45`	Mutation policy disabled for query mode
E4 · export	Derive dates, set filters, trigger export	Delivered; 2026-09-01 through 2026-09-30	Read filters back before export
E5 · ambiguity	Refuse unsafe identity resolution	No diff; failed; both identifiers in reason	Exactly-one-match invariant

State score

Use `/__reset` before each run and `/__check` after completion. Record each named check, not just aggregate pass/fail.

Behavior score

Assert action count, no input after terminal state, deadline compliance, retries, and unique desktop ownership.

Artifact score

Parse terminal status and summary: exact `respuesta:` line rules, ordered semicolon values, required reason keys, no secrets.

Domain-shift test before the incident

Run the same capability suite against a locally reskinned mock that changes navigation position, colors, labels, and entity vocabulary while preserving task semantics. Then run a static audit that fails if production code contains known app labels, entity IDs, or pixel coordinates. This does not predict the clinic UI; it proves the system is not dependent on SIGECOM.

Operational metrics

task success rate · false-success rate · extra-action rate · duplicate-write rate · median actions/task · p95 step latency · grounding retry rate · model cost/task · deadline cancellations

Release gate. No hidden evaluation until all five public tasks pass from clean resets twice, the summary parser passes, the no-hardcoding audit passes, and a forced cancellation produces zero VNC events after revocation.

Correctness first; sophistication only where it changes the score

Decision	Chosen approach	Rejected alternative	Reason
Driver	Small custom RFB client over supplied WebSocket	Hosted computer-use service	Meets restriction; full timing/cancellation control.
Planning	One semantic action at a time	Long end-to-end macro	Contains errors and adapts to popups/layout changes.
Evidence	Fresh screenshot + postcondition	Trust tool acknowledgement	Legacy repaint makes acknowledgement insufficient.
State	Simple durable registry + isolated worker	In-process fire-and-forget only	Pollable status and reliable cancellation matter.
Output	Deterministic templates	Free-form model summary	Formatting and secret exclusion are graded.

Validate the irreversible boundaries first

Build sequence

P0 · prove

RFB handshake, full screenshot, pointer and key events, stable-frame wait, forced disconnect.

P1 · contract

FastAPI schema/auth, under-two-second task creation, status polling, terminal state, redaction.

P2 · loop

Structured visual planner, policy gate, one-action execution, postcondition verifier, bounded recovery.

P3 · coverage

Create, update, query/count, export, ambiguity refusal; deterministic summary composer.

P4 · harden

SSRF controls, rate/cost limits, worker isolation, cancellation test, reskinned mock, deployment smoke test.

Top residual risks

Risk	Likelihood / impact	Mitigation
Visual grounder misses a small control	Medium / high	Zoom, neighborhood evidence, keyboard fallback, one re-grounded retry.
Hidden app expresses a capability differently	Medium / high	Explore-first semantic inventory; role-based rather than label-based mapping.
Model latency consumes budget	Medium / high	Call ceilings, compressed state, cached inventory, hard time reservation.
Write succeeds but confirmation is absent	Low / high	Search/read-back verification; never repeat a potentially committed write blindly.

Design grounded in protocol and security practice

1. **Remote Framebuffer Protocol.** RFC 6143 documents RFB's demand-driven framebuffer updates, incremental requests, input events, and security model. https://datatracker.ietf.org/doc/html/rfc6143
2. **OWASP API Security Top 10 (2023).** The design maps especially to unrestricted resource consumption, server-side request forgery, broken authentication, and unsafe downstream consumption. https://api-security.owasp.org/editions/2023/en/0x11-t10/
3. **OWASP SSRF Prevention Cheat Sheet.** Supports strict destination allowlisting, redirect refusal, DNS/IP validation, connect-time enforcement, and network-layer defense in depth. https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
4. **OpenTelemetry: Handling sensitive data.** Supports data minimization and filtering/redaction of credentials, personal data, health data, and tokens from telemetry. https://opentelemetry.io/docs/security/handling-sensitive-data/
5. **FastAPI Background Tasks.** Establishes the asynchronous response pattern; the solution adds a separate worker because these jobs are long-running and require explicit cancellation and status. https://fastapi.tiangolo.com/reference/background/
6. **Anthropic Computer Use documentation.** Supports the observe-act loop, sequential execution, screenshot verification after actions, resolution discipline, and treating screen content as a prompt-injection surface. The implementation remains provider-neutral and uses no hosted computer-operation service. https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool?ref=explainx
7. **NIST AI Risk Management Framework.** Provides the broader rationale for integrating trustworthiness into design, development, evaluation, and operation rather than treating it as a post-build checklist. https://www.nist.gov/itl/ai-risk-management-framework

Challenge source of truth. The supplied README and E1–E5 practice files define the interface contract, screen behavior, scoring semantics, and public examples. They are treated as authoritative over generic platform guidance.

The design is intentionally simple at the edge and strict at the moment of action: understand broadly, act narrowly, verify visibly, stop definitively.